

ReviewDAOImpl.java

```
package com.harisudhan.harisudhanmart.dao;

import com.harisudhan.harisudhanmart.model.Review;
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;
import java.sql.Timestamp;
import java.util.ArrayList;
import java.util.List;
import javax.sql.DataSource;

public class ReviewDAOImpl implements ReviewDAO {

    private final DataSource dataSource;

    public ReviewDAOImpl(DataSource dataSource) {
        this.dataSource = dataSource;
    }

    @Override
    public Review create(Review review) throws SQLException {
        String sql = "INSERT INTO reviews (product_id, user_id, rating, comment) VALUES (?, ?, ?, ?)";
        try (Connection conn = dataSource.getConnection();
            PreparedStatement ps = conn.prepareStatement(sql, Statement.RETURN_GENERATED_KEYS)) {
            ps.setLong(1, review.getProductid());
            ps.setLong(2, review.getUserid());
            ps.setInt(3, review.getRating());
            ps.setString(4, review.getComment());
            ps.executeUpdate();
            try (ResultSet keys = ps.getGeneratedKeys()) {
                if (keys.next()) {
                    review.setId(keys.getLong(1));
                }
            }
        }
        return review;
    }

    @Override
    public List<Review> findByProduct(long productId) throws SQLException {
        String sql = "SELECT * FROM reviews WHERE product_id = ? ORDER BY created_at DESC";
        try (Connection conn = dataSource.getConnection();
            PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setLong(1, productId);
            try (ResultSet rs = ps.executeQuery()) {
                List<Review> reviews = new ArrayList<>();
                while (rs.next()) {
                    Review r = new Review();
                    r.setId(rs.getLong("id"));
                    r.setProductId(rs.getLong("product_id"));
                    r.setUserId(rs.getLong("user_id"));
                    r.setRating(rs.getInt("rating"));
                    r.setComment(rs.getString("comment"));
                    Timestamp ts = rs.getTimestamp("created_at");
                    if (ts != null) {
                        r.setCreatedAt(ts.toLocalDateTime());
                    }
                    reviews.add(r);
                }
            }
        }
        return reviews;
    }
}
```

```

    }
}

@Override
public boolean buyerPurchasedProduct(long userId, long productId) throws SQLException {
    String sql = "SELECT COUNT(*) FROM order_items oi "
        + "JOIN orders o ON o.id = oi.order_id "
        + "WHERE o.buyer_id = ? AND oi.product_id = ? AND o.status = 'DELIVERED'";
    try (Connection conn = dataSource.getConnection();
        PreparedStatement ps = conn.prepareStatement(sql)) {
        ps.setLong(1, userId);
        ps.setLong(2, productId);
        try (ResultSet rs = ps.executeQuery()) {
            return rs.next() && rs.getInt(1) > 0;
        }
    }
}
}

```